

Informe del proyecto

Home Catalogue

Integrantes: Nicolas Moya y Nicolás Sobarzo

Herramientas:

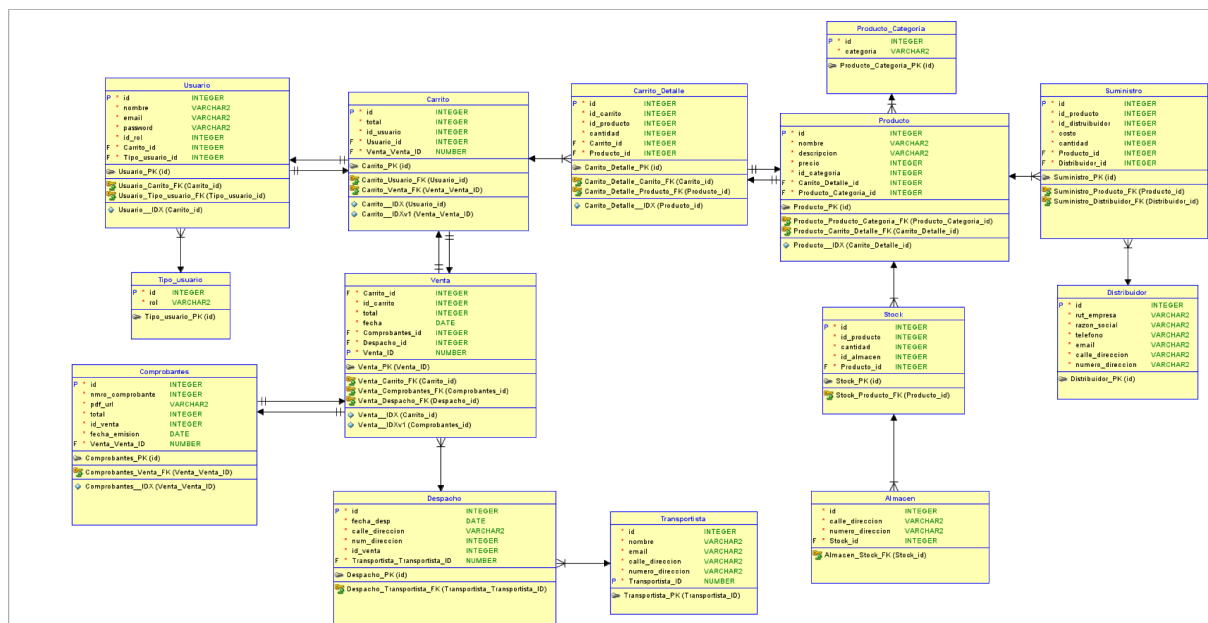
- Spring Web
- Spring Reactive Web
- Spring Data JPA
- My SQL driver
- Lombok
- Validation
- Gateway
- Github
- Github Desktop
- Visual Studio Code

1. Contexto del proyecto	3
2. Modelo Entidad-Relación	3
Servicio de autenticación	3
Implementación por capas	3
Swagger	4
Testing	4

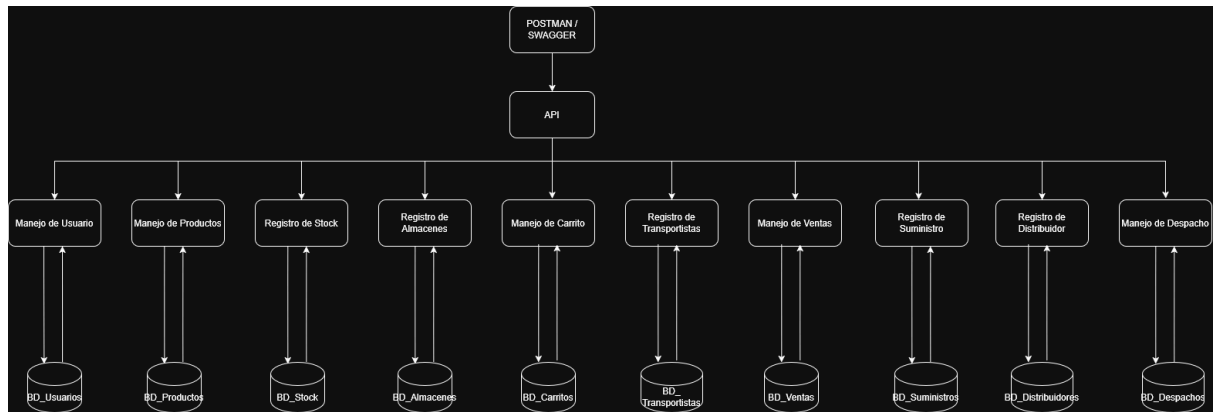
1.Contexto del proyecto

El cliente posee una empresa independiente con un catálogo de artículos para el hogar. Su catálogo es extenso y diverso, por lo que su principal desafío radica en su modelo operativo actual, que es 100% manual y tradicional. Dada esta falta de digitalización, nuestra propuesta busca automatizar integralmente estos procesos mediante el desarrollo de “Home Catalogue”, garantizando una gestión de inventario y ventas mucho más eficiente, reduciendo los errores humanos.

2.Modelo Entidad-Relación



3.Diagrama de microservicios



4. Servicio de autenticación

- I. Un servicio de autenticación es un servicio encargado de resguardar la seguridad del software verificando que el usuario operándolo tenga los permisos necesarios para realizar ciertas operaciones.
- II. Su implementación implica la creación de un proyecto Maven nuevo con su propia base de datos y modificaciones en el gateway.

5. Implementación por capas

En esta sección se detalla el código y ejecución de los métodos del microservicio de gestión de usuarios.

TipoUsuarioController:

```

package com.servicio.usuario.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.servicio.usuario.model.TipoUsuario;
import com.servicio.usuario.repository.TipoUsuarioRepository;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@CrossOrigin(origins="*")
@RestController
@RequestMapping("/api/v1/usuarios/tipos")
@Tag(name = "Tipos de usuario", description = "Operaciones relacionadas con la gestión de los tipos de usuario")
public class TipoUsuarioController {

    @Autowired
    private TipoUsuarioRepository tipoUsuarioRepository;

    @Operation(summary = "Obtener todos los tipos de usuarios", description = "Retorna una lista completa de los tipos de usuarios")
    @GetMapping
    public List<TipoUsuario> listar(){
        return tipoUsuarioRepository.findAll();
    }

    @Operation(summary = "Crear un tipo de usuario", description = "Crea un tipo de usuario en base a los datos ingresados")
    @PostMapping TipoUsuario guardar(@Valid @RequestBody TipoUsuario tipo){
        return tipoUsuarioRepository.save(tipo);
    }

    @Operation(summary = "Borrar un tipo de usuario", description = "Borra el tipo de usuario cuya Id coincida con la que fue ingresada")
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id){
        tipoUsuarioRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

```

UsuarioController:

```

import jakarta.validation.Valid;

@CrossOrigin(origins="*")
@RestController
@RequestMapping("/api/v1/usuarios")
@Tag(name = "Usuarios", description = "Operaciones relacionadas con la gestión de usuarios")
public class UsuarioController {

    @Autowired
    private UsuarioService usuarioService;

    @Operation(summary = "Obtener todos los usuarios", description = "Retorna una lista completa de los usuarios")
    @GetMapping
    public List<Usuario> listar(){
        return usuarioService.listarUsuarios();
    }

    @Operation(summary = "Obtener un usuario mediante su Id", description = "Retorna el usuario cuya Id coincide con la ingresada")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "Paciente encontrado"),
        @ApiResponse(responseCode = "404", description = "No existe el paciente que busca", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
    })
    @GetMapping("/{id}")
    public ResponseEntity<Usuario> buscarPorId(@PathVariable Long id) {
        return usuarioService.buscarPorId(id)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
    }

    @Operation(summary = "Crear un usuario", description = "Crea un usuario en base a los datos ingresados")
    @PostMapping ResponseEntity<Usuario> guardar(@Valid @RequestBody Usuario usuario){
        return ResponseEntity.ok(usuarioService.guardarUsuario(usuario));
    }

    @Operation(summary = "Borrar un usuario", description = "Borra el usuario cuya Id coincida con la que fue ingresada")
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id){
        usuarioService.eliminar(id);
        return ResponseEntity.noContent().build();
    }

    @Operation(summary = "Obtener un usuario mediante su run", description = "Retorna el usuario cuyo run coincide con el ingresado")
    @GetMapping("/{run/{run}")
    public ResponseEntity<Usuario> buscarPorRun(@PathVariable String run){
        return usuarioService.encontrarPorRun(run)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @Operation(summary = "Actualizar un usuario", description = "Actualiza un usuario en base a los datos ingresados")
    @PutMapping("/{id}")
    public ResponseEntity<Usuario> modificar(@Valid @PathVariable Long id, @RequestBody Usuario usuario){
        try {
            ResponseEntity<Usuario> user = usuarioService.buscarPorId(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());

            user.getBody().setEmail(usuario.getEmail());
            user.getBody().setPassword(usuario.getPassword());

            usuarioService.guardarUsuario(user.getBody());

            return user;
        } catch (Exception e){
            return ResponseEntity.notFound().build();
        }
    }
}

```

Método POST utilizado para crear un tipo de usuario:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:9080/api/v1/usuarios/tipos
- Body (JSON):**

```

{
  "rol": "Cliente"
}

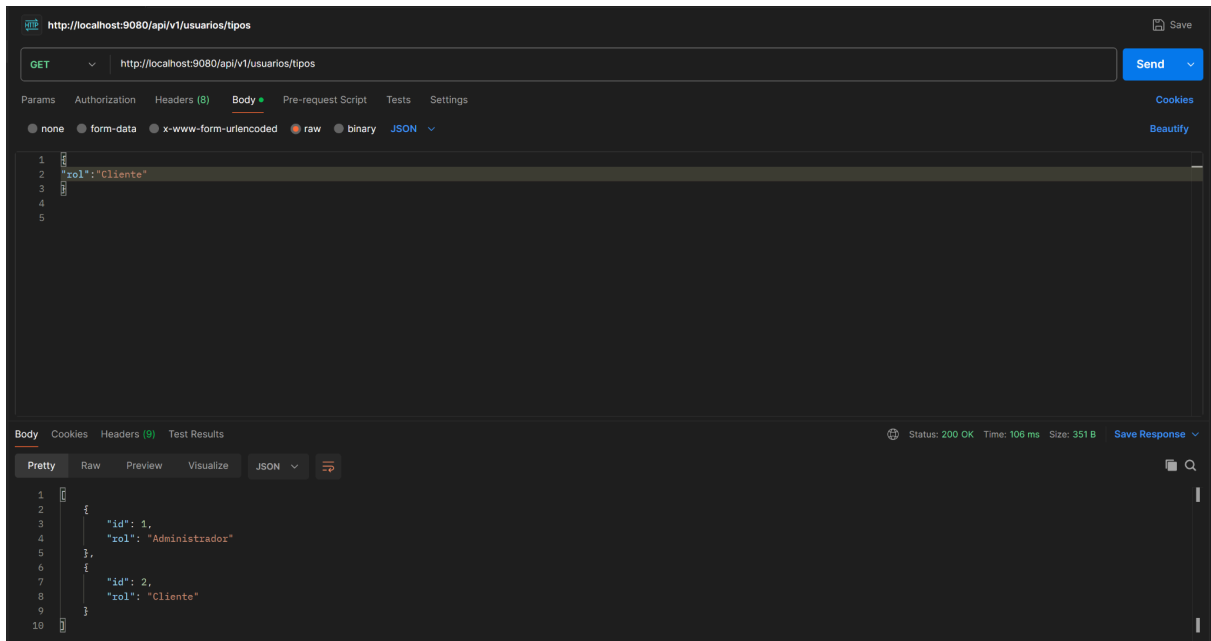
```
- Status:** 200 OK
- Time:** 533 ms
- Size:** 324 B
- Response Body (JSON):**

```

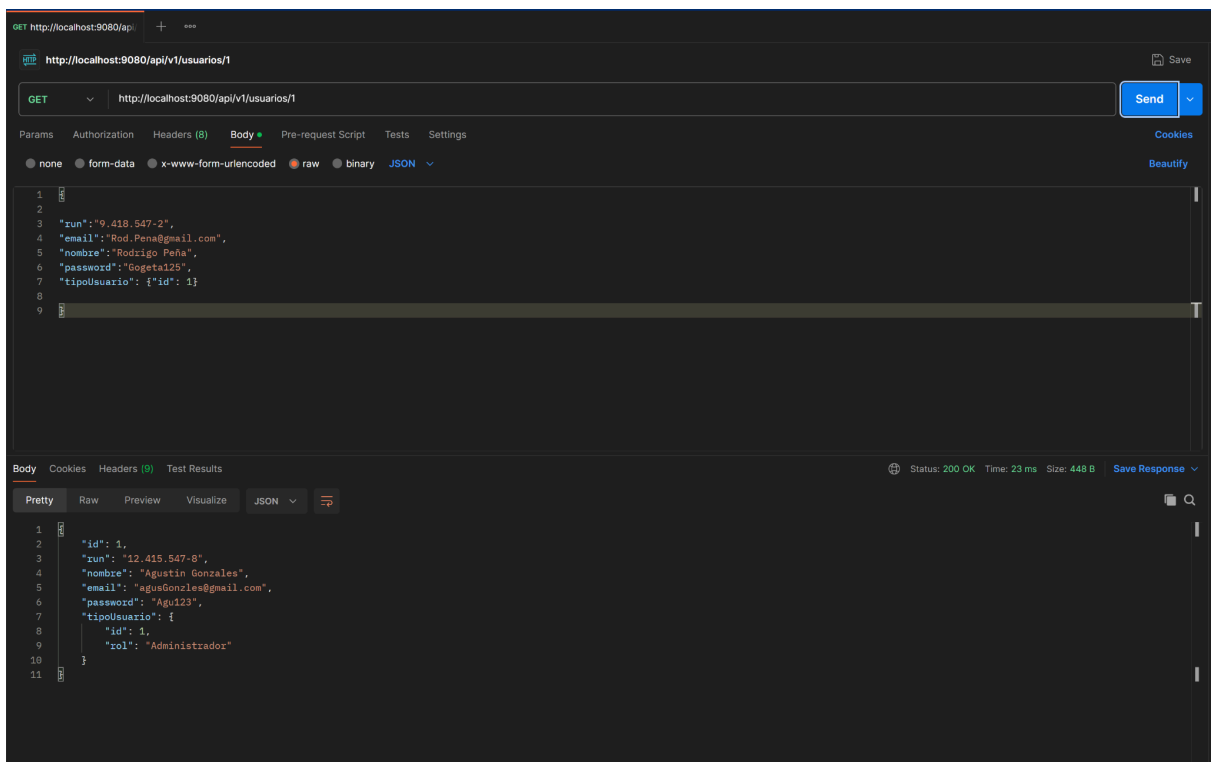
{
  "id": 1,
  "rol": "Administrador"
}

```

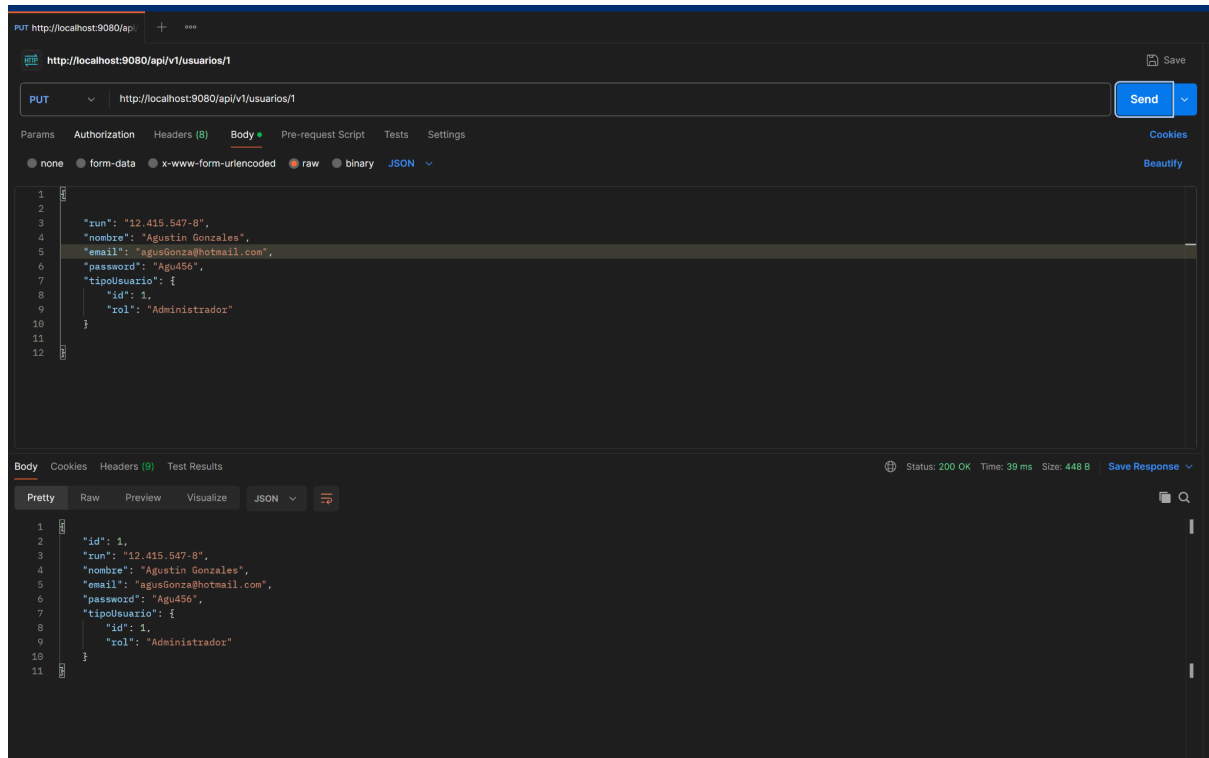
Método GET utilizado para traer la lista de los tipos de usuario



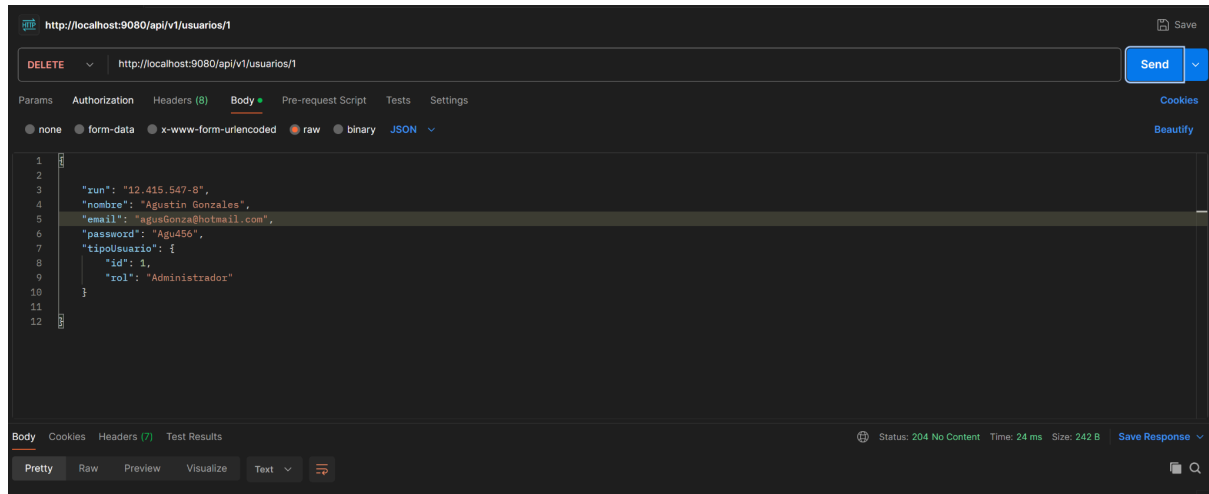
Método GET(id) utilizado para obtener los datos de un usuario con una id en particular

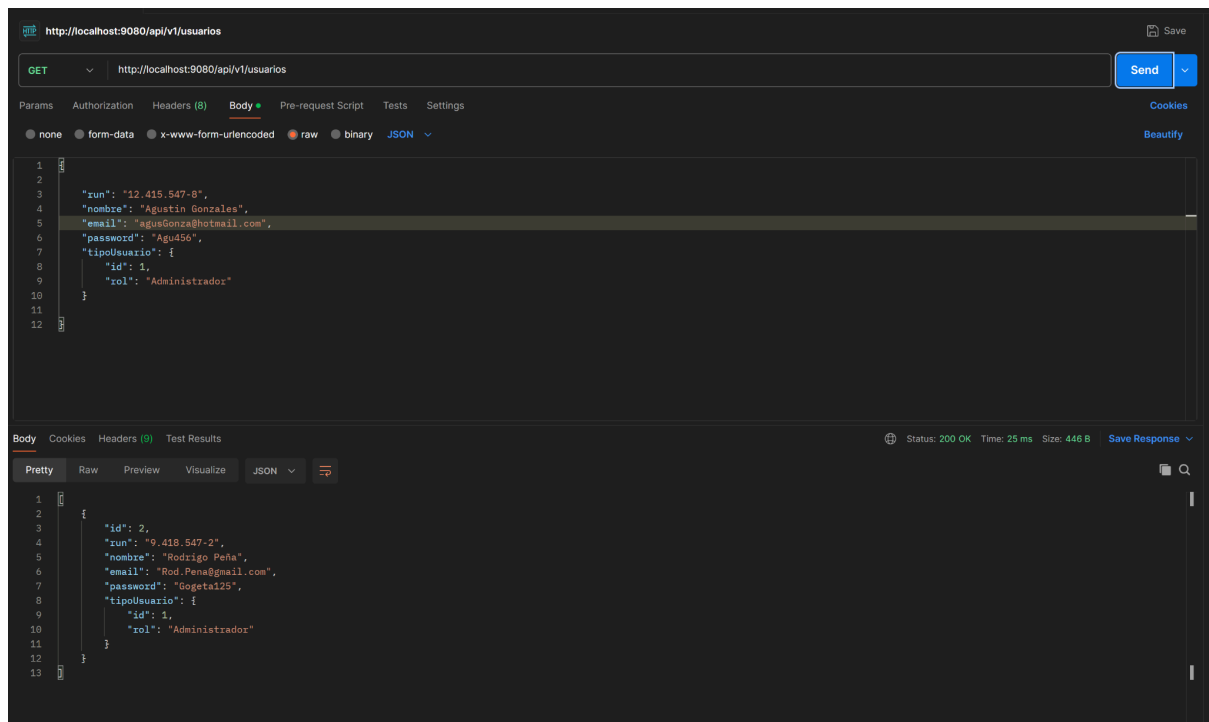


Método PUT(id) utilizado para actualizar los datos de un usuario con una id en particular



Método DELETE(id) utilizado para eliminar un usuario con una id en particular





II. La comunicación entre microservicios se realiza mediante una clase llamada `WebClientConfig`, la cual permite obtener los datos de otro microservicio mediante un método en el service. Dicho método debe colocarse en el microservicio que va a recibir datos, no en el que los envía. Este método es utilizado en otros métodos del service para enriquecer los datos con lo que se obtiene externamente. Para el correcto funcionamiento de este método es necesario especificar en el modelo del microservicio un atributo y un objeto que aludan al dato que se quiere obtener y al objeto del que proviene, respectivamente

6. Swagger

7. Testing

El testing o pruebas unitarias consiste en ejecutar un sistema o componente bajo condiciones controladas para verificar que funciona según lo esperado, identificando errores y asegurar la calidad del producto.

- **Objetivo:** La importancia de implementar las pruebas unitarias es fundamental para asegurar que cada “pieza” de nuestro proyecto, es decir, servicios y lógica de negocio funcione correctamente. Y logramos esto de una forma aislada sin depender de que la base de datos o el otro microservicio estén encendidos.

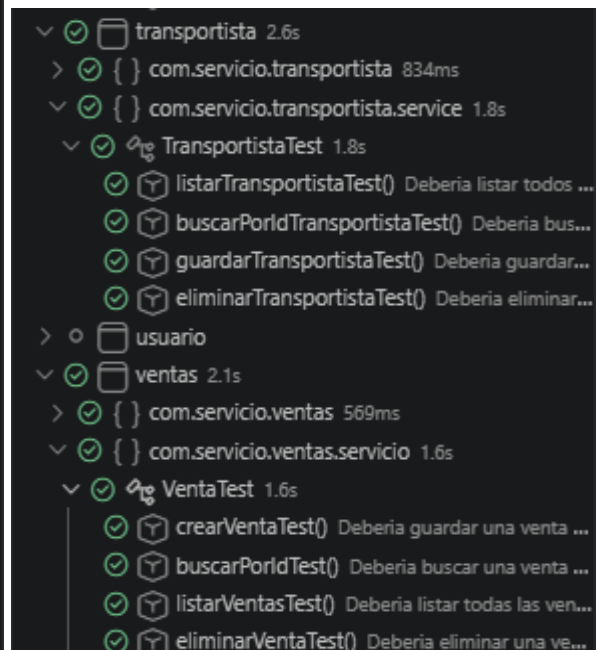
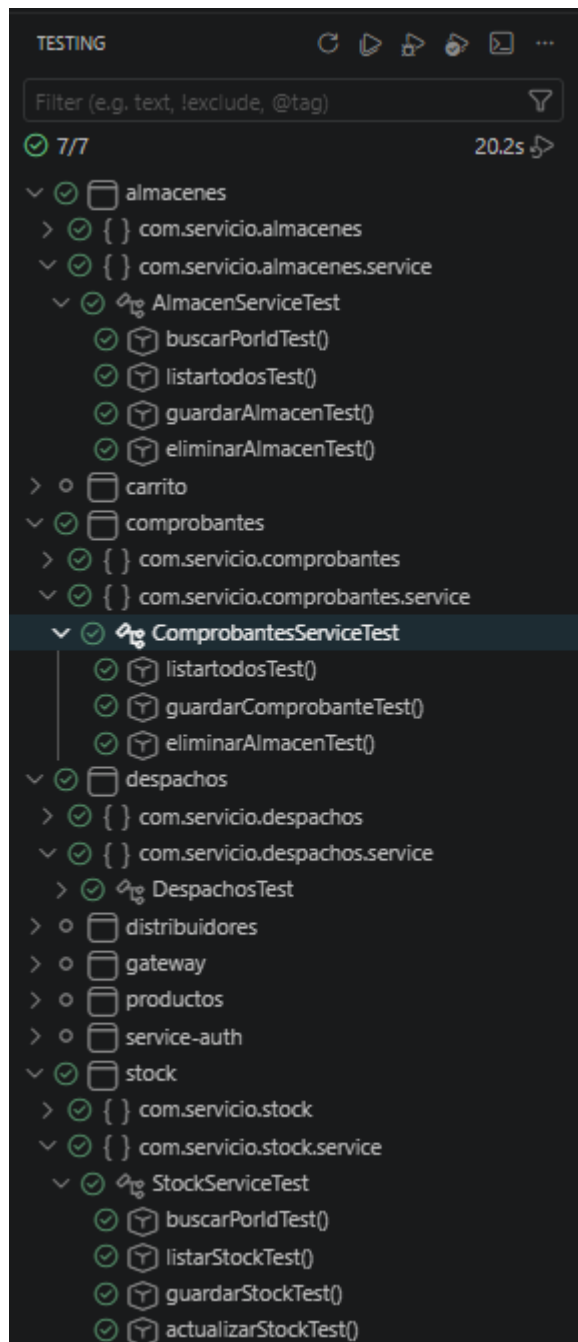
- Prevención de regresiones: Asegura que los nuevos cambios en el código no rompan funcionalidades que ya servían.
- Velocidad: Son extremadamente rápidas (milisegundos)
- Ejemplo: Un buen set de pruebas sirve como un manual técnico que explica de forma exacta el comportamiento esperado por el sistema.

Las Pruebas Unitarias (AAA)

Son el primer nivel de pruebas y se centran en la unidad más pequeña de código de forma aislada. Su característica principal es que son extremadamente rápidas, ejecutándose en cuestión de milisegundos. Y para lograr el aislamiento, si el código depende de recursos externos como una base de datos o una API, se utilizan “dobles de prueba” y estos se llaman JUnit 5 y Mockito son las herramientas que nos permiten simular el comportamiento de los repositorios o de WebClient.

Para estructurar estas pruebas de forma limpia y estándar, se utiliza el patrón AAA (Arrange, Act, Assert) o su equivalente Preparación, Ejecución y Verificación:

1. Arrange (Preparación): Se configura el escenario de la prueba. Aquí se crean los objetos necesarios, los datos de entrada y se define el comportamiento de los simulacros (*mocks*) utilizando palabras clave como `when(...)`.
2. Act (Ejecución): Se invoca el método real de la lógica de negocio que se desea poner a prueba.
3. Assert (Verificación): Se comprueba que el resultado obtenido coincide con el esperado utilizando aserciones (`assertEquals`, `assertNotNull`) y se valida que los métodos simulados se hayan llamado de forma correcta (`verify`).



Evidencias de cada método implementado en cada capa de testeo explicando lo que se esta haciendo:

```
J StockServiceTest.java X
Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest

1  package com.servicio.stock.service;
2
3  import static org.junit.jupiter.api.Assertions.assertEquals;
4  import static org.junit.jupiter.api.Assertions.assertNotNull;
5  import static org.mockito.ArgumentMatchers.any;
6  import static org.mockito.ArgumentMatchers.anyString;
7  import static org.mockito.Mockito.times;
8  import static org.mockito.Mockito.verify;
9  import static org.mockito.Mockito.when;
10
11 import java.util.List;
12 import java.util.Optional;
13
14 import org.junit.jupiter.api.DisplayName;
15 import org.junit.jupiter.api.Test;
16 import org.junit.jupiter.api.extension.ExtendWith;
17 import org.mockito.InjectMocks;
18 import org.mockito.Mock;
19 import org.mockito.Mockito;
20 import org.mockito.junit.jupiter.MockitoExtension;
21 import org.springframework.web.reactive.function.client.WebClient;
22
23 import com.servicio.stock.dto.AlmacenDTO;
24 import com.servicio.stock.dto.CategoriaDTO;
25 import com.servicio.stock.dto.ProductoDTO;
26 import com.servicio.stock.dto.StockDTO;
27 import com.servicio.stock.model.Stock;
28 import com.servicio.stock.repository.StockRepository;
29
30 import reactor.core.publisher.Mono;
31
32 @ExtendWith(MockitoExtension.class)
33 public class StockServiceTest {
34
35     @Mock
36     private StockRepository stockRepository;
37
38     @Mock
39     private WebClient.Builder webClientBuilder;
40
41     @InjectMocks
42     private StockService stockService;
43 }
```

La Anotación `@ExtendWith(MockitoExtension.class)`: Es el motor que activa Mockito en JUnit 5. Sin esta línea, las anotaciones `@Mock` de abajo no funcionarían y te darían un error de puntero nulo (`NullPointerException`).

La Anotación `@Mock` en `StockRepository`: Crea un objeto simulado de tu base de datos. Sirve para que cuando ejecutes la prueba, el sistema no intente conectarse a una base de datos real o física, donde aísla la lógica del servicio de manera pura.

La Anotación `@Mock` en `WebClient.Builder`: Crea un objeto simulado (falso) del cliente HTTP que usas para conectarte con otros microservicios (como el de Productos o Almacenes).

La Anotación @InjectMocks en StockService: Esta es la anotación más importante para el

```
J StockServiceTest.java X
Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest > listarStockTest()
33 public class StockServiceTest {
85 void listarStockTest(){
119 assertEquals(expected: "Hogar", dtoResultados.getDatosProducto().getProductoCategoria().getCategoria());
120 verify(stockRepository, times(wantedNumberOfInvocations: 1)).findAll();
121 }
122
123 @Test
124 @DisplayName("Deberia guardar el stock correctamente")
125 void guardarStockTest(){
126     Stock stock = new Stock();
127     stock.setCantidad(cantidad: 50);
128     stock.setIdProducto(idProducto: 1L);
129     stock.setIdAlmacen(idAlmacen: 1L);
130     stock.setDatosAlmacen(stock.getDatosAlmacen());
131     stock.setDatosProducto(stock.getDatosProducto());
132     when(stockRepository.save(any(type: Stock.class))).thenReturn(invocation ->{
133         Stock a = invocation.getArgument(index: 0);
134         a.setId(id: 1L);
135         return a;
136     });
137     Stock resultado = stockService.crearStock(stock);
138     assertNotNull(resultado);
139     assertEquals(expected: 50, resultado.getCantidad());
140     assertEquals(expected: 1L, resultado.getIdProducto());
141     assertEquals(expected: 1L, resultado.getIdAlmacen());
142     assertNotNull(resultado.getDatosAlmacen());
143     assertNotNull(resultado.getDatosProducto());
144     verify(stockRepository, times(wantedNumberOfInvocations: 1)).save(stock);
145 }
146
147
148 @Test
149 @DisplayName("Deberia actualizar un stock por ID correctamente")
150 void actualizarStockTest(){
151     Stock stock = new Stock();
152     stock.setId(id: 1L);
153     stock.setCantidad(cantidad: 50);
154     stock.setIdAlmacen(idAlmacen: 1L);
155     stock.setIdProducto(idProducto: 1L);
```

```
J StockServiceTest.java X
Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest > listarStockTest()
33 public class StockServiceTest {
83 @test
84 @DisplayName("Deberia listar el despacho correctamente")
85 void listarStockTest(){
86     Stock stock = new Stock();
87     stock.setId(id: 1L);
88     stock.setCantidad(cantidad: 50);
89     stock.setIdProducto(idProducto: 1L);
90     stock.setIdAlmacen(idAlmacen: 1L);
91     List<Stock> listaStock = List.of(stock);
92     when(stockRepository.findAll()).thenReturn(listaStock);
93     AlmacenDTO almacenDTO = new AlmacenDTO(id: 1L, calleDireccion: "Av Pajaritos", numeroDireccion: "1122");
94     CategoriaDTO categoriaDTO = new CategoriaDTO(id: 1L, categoria: "Hogar");
95     ProductoDTO productoDTO = new ProductoDTO(id: 1L, nombre: "Estante Negro", descripcion: "180X90X60", precio: 21000, categoriaDTO );
96     WebClient webClient = Mockito.mock(classToMock: WebClient.class);
97     WebClient.RequestHeadersUriSpec uriSpec = Mockito.mock(classToMock: WebClient.RequestHeadersUriSpec.class);
98     WebClient.RequestHeadersSpec headersSpec = Mockito.mock(classToMock: WebClient.RequestHeadersUriSpec.class);
99     WebClient.ResponseSpec responseSpec = Mockito.mock(classToMock: WebClient.ResponseSpec.class);
100     when(webClientBuilder.build()).thenReturn(webClient);
101     when(webClient.get()).thenReturn(uriSpec);
102     when(uriSpec.uri(anyString())).thenReturn(headersSpec);
103     when(headersSpec.retrieve()).thenReturn(responseSpec);
104     when(responseSpec.bodyToMono(any(type: Class.class)))
105         .thenReturn(Mono.just(productoDTO))
106         .thenReturn(Mono.just(almacenDTO));
107     List<StockDTO> resultado = stockService.listar();
108     assertNotNull(resultado, message: "La lista de stock no debe ser nula");
109     assertEquals(expected: 1, resultado.size(), message: "Deberia retornar examente un elemento");
110     StockDTO dtoResultados = resultado.get(0);
111     assertEquals(expected: 1L, dtoResultados.getId());
112     assertEquals(expected: 50, dtoResultados.getCantidad());
113     assertEquals(expected: 1L, dtoResultados.getIdProducto());
114     assertEquals(expected: 1L, dtoResultados.getIdAlmacen());
115     assertNotNull(dtoResultados.getDatosAlmacen(), message: "Los datos del almacen no deben ser nulos");
116     assertEquals(expected: "Av Pajaritos", dtoResultados.getDatosAlmacen().getCalleDireccion());
117     assertNotNull(dtoResultados.getDatosProducto(), message: "Los datos del producto no deben ser nulos");
118     assertEquals(expected: "Estante Negro", dtoResultados.getDatosProducto().getNombre());
119     assertEquals(expected: "Hogar", dtoResultados.getDatosProducto().getProductoCategoria().getCategoria());
120     verify(stockRepository, times(wantedNumberOfInvocations: 1)).findAll();
```

J StockServiceTest.java X

Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest

```
33 public class StockServiceTest {
44     @Test
45     @DisplayName("Deberia buscar por Id el stock correctamente")
46     void buscarPorIdTest(){
47         Stock stock = new Stock();
48         stock.setId(id: 1L);
49         stock.setCantidad(cantidad: 50);
50         stock.setIdProducto(idProducto: 1L);
51         stock.setIdAlmacen(idAlmacen: 1L);
52
53         when(stockRepository.findById(id: 1L)).thenReturn(Optional.of(stock));
54
55         AlmacenDTO almacenDTO = new AlmacenDTO(id: 1L, calleDireccion: "Av Pajaritos", numeroDireccion: "1122");
56         CategoriaDTO categoriaDTO = new CategoriaDTO(id: 1L, categoria: "Hogar");
57         ProductoDTO productoDTO = new ProductoDTO(id: 1L, nombre: "Estante Negro", descripcion: "180x90x60", precio: 21000, categoriaDTO);
58
59         WebClient webClient = Mockito.mock(classToMock: WebClient.class);
60         WebClient.RequestHeadersUriSpec uriSpec = Mockito.mock(classToMock: WebClient.RequestHeadersUriSpec.class);
61         WebClient.RequestHeadersSpec headersSpec = Mockito.mock(classToMock: WebClient.RequestHeadersUriSpec.class);
62         WebClient.ResponseSpec responseSpec = Mockito.mock(classToMock: WebClient.ResponseSpec.class);
63
64         when(webClientBuilder.build()).thenReturn(webClient);
65         when(webClient.get()).thenReturn(uriSpec);
66         when(uriSpec.uri(anyString())).thenReturn(headersSpec);
67         when(headersSpec.retrieve()).thenReturn(responseSpec);
68
69         when(responseSpec.bodyToMono(any(type: Class.class)))
70             .thenReturn(Mono.just(productoDTO))
71             .thenReturn(Mono.just(almacenDTO));
72
73         StockDTO resultado = stockService.buscarPorId(id: 1L);
74
75         assertNotNull(resultado, message: "El StockDTO no deberia ser nulo");
76         assertEquals(expected: 1L, resultado.getId());
77         assertEquals(expected: 50, resultado.getCantidad());
78         assertEquals(expected: "Estante Negro", resultado.getDatosProducto().getNombre());
79         assertEquals(expected: "Av Pajaritos", resultado.getDatosAlmacen().getCalleDireccion());
80         verify(stockRepository, times(wantedNumberOfInvocations: 1)).findById(id: 1L);
81     }
}
```

```
StockServiceTest.java X
Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest > listarStockTest()
33 public class StockServiceTest {
125     void guardarStockTest(){
144         verify(stockRepository, times(wantedNumberOfInvocations: 1)).save(stock);
145     }
146
147
148     @Test
149     @DisplayName("Deberia actualizar un stock por ID correctamente")
150     void actualizarStockTest(){
151         Stock stock = new Stock();
152         stock.setId(id: 1L);
153         stock.setCantidad(cantidad: 50);
154         stock.setIdAlmacen(idAlmacen: 1L);
155         stock.setIdProducto(idProducto: 1L);
156
157         when(stockRepository.findById(id: 1L)).thenReturn(Optional.of(stock));
158         when(stockRepository.save(any(type: Stock.class))).thenAnswer(invocation ->{
159             Stock a = invocation.getArgument(index: 0);
160             return a;
161         });
162         Stock resultado = stockService.crearStock(stock);
163
164         assertNotNull(resultado, message: "El stock actualizado no deberia ser null");
165         assertEquals(expected: 1L, resultado.getId());
166         assertEquals(expected: 50, resultado.getCantidad());
167         assertEquals(expected: 1L, resultado.getIdProducto());
168         assertEquals(expected: 1L, resultado.getIdAlmacen());
169
170         assertNotNull(resultado.getDatosAlmacen());
171         assertNotNull(resultado.getDatosProducto());
172         verify(stockRepository, times(wantedNumberOfInvocations: 1)).save(stock);
173     }
174
175
176
177
178
179
180
181
```

servicio real. Le dice a Mockito:

```
J StockServiceTest.java X
Microservicio - Manejo de Stock > stock > src > test > java > com > servicio > stock > service > J StockServiceTest.java > StockServiceTest > listarStockTest()
33 public class StockServiceTest {
150 void actualizarStockTest(){
170     assertNotNull(resultado.getDatosAlmacen());
171     assertNotNull(resultado.getDatosProducto());
172     verify(stockRepository, times(wantedNumberOfInvocations: 1)).save(stock);
173 }
174
175
176
177
178
179
180
181
182
183
184
185 @Test
186 @DisplayName("Deberia eliminar un stock por ID correctamente")
187 void eliminarStockTest(){
188     Long id = 1L;
189     Mockito.doNothing().when(stockRepository).deleteById(id);
190     stockService.eliminarStock(id);
191     verify(stockRepository, times(wantedNumberOfInvocations: 1)).deleteById(id);
192 }
193
194
195
196
197
198
199
200
201
202
203 }
204
```

Método buscarPorIdTest()

- Qué se está testeando: La integración correcta entre la base de datos local (simulada) y las consultas externas mediante WebClient. Valida que cuando buscas un stock por su ID (1L), el servicio recupere el registro base y desencadene llamadas HTTP reactivas ordenadas: la primera llamada para recuperar la información del producto ("Estante Negro") y la segunda para la dirección del almacén ("Av Pajaritos").

Explicación del Patrón AAA en la Imagen:

- Arrange: Creas el stock base, defines que el repositorio devuelva ese stock (findById(1L)) y configuras la cadena simulada de WebClient para retornar consecutivamente los objetos productoDTO y almacenDTO.
- Act: Se ejecuta el método real del servicio buscarPorId(1L).
- Assert: Compruebas con aserciones que los datos resultantes no sean nulos, que pertenezcan al ID correcto, y que el mapeo reactivo unificara los nombres del producto y la calle del almacén correctamente.

2. Método listarStockTest()

- Qué se está testeando: El comportamiento del método listar() al manejar colecciones de datos y resolver dependencias asíncronas para cada elemento. Comprueba que el servicio extraiga correctamente la lista completa desde el repositorio (findAll()), itere sobre ella, y consuma la cadena reactiva de WebClient para poblar los campos anidados de todos los DTOs en la lista final.

Explicación del Patrón AAA en la Imagen:

- Arrange : Envuelves tu entidad Stock dentro de una lista inmutable (List.of(stock)) y preparas los mocks de datos externos para simular el consumo HTTP reactivo.
- Act : Invocas la ejecución de stockService.listar().
- Assert Validas que la lista contenga exactamente 1 elemento (size() == 1), extraes el DTO y verificas la correcta resolución de jerarquías más profundas como la categoría del producto ("Hogar").

3. Método guardarStockTest()

- Qué se está testeando: El proceso transaccional de creación de un registro de stock. Específicamente, comprueba que el objeto de entrada mantenga la integridad de sus propiedades en el flujo del servicio y que la persistencia simule con éxito la asignación dinámica de un identificador único autoincremental (1L) a través de la respuesta de la base de datos.

Explicación del Patrón AAA en la Imagen:

- Arrange Configuras una entidad sin ID definido en un inicio. Utilizas .thenAnswer(...) para interceptar la llamada de guardado del repositorio, tomar el argumento enviado en tiempo de ejecución (invocation.getArgument(0)) y mutarlo agregándole de forma segura el ID simulado 1L.
- Act Invocas la creación real con stockService.crearStock(stock).
- Assert Compruebas que el objeto retornado tenga guardados los valores originales (cantidad 50, ID de producto 1L, ID de almacén 1L) y su nuevo ID asignado.

4. Método actualizarStockTest()

- Qué se está testeando: La lógica de modificación de un stock. El test confirma que el servicio realice primero una validación previa buscando si el registro existe en la base de datos a través de su ID (findById) antes de proceder a guardar los nuevos cambios sobre la entidad.

Explicación del Patrón AAA en la Imagen:

- Arrange Preparas el escenario forzando a que findById(1L) responda con un registro válido (Optional.of(stock)) y que el método save complete el almacenamiento retornando la misma entidad modificada.
- Act Se invoca la función de actualización en el servicio.
- Assert: Validas mediante aserciones la persistencia del objeto resultante y aseguras que el repositorio ejecutó el guardado con los datos correctos.

5. Método eliminarStockTest()

- Qué se está testeando: El flujo de borrado lógico o físico por ID. Dado que el borrado de datos en Spring Data JPA suele ser una operación de tipo void (no regresa un objeto), este test no comprueba un valor devuelto, sino que testea el comportamiento de la comunicación. Garantiza que el microservicio propague la petición de baja de manera limpia y que el método del repositorio se ejecute exactamente una vez con el ID correcto.

Explicación del Patrón AAA

Arrange: Declaras el identificador objetivo (id = 1L) y empleas Mockito.doNothing() para indicarle al mock del repositorio que no realice ninguna operación destructiva real ni levante excepciones al recibir la llamada.

- Act Ejecutas la llamada de eliminación en el servicio.

- Assert : Utilizas el método de verificación conductual `verify(..., times(1)).deleteById(id)` para validar técnicamente que la capa de negocio llamó al repositorio con el ID exacto que se deseaba remover.